Forge: Gym Tracker

Project summary: requirements, architecture and design

Forge is a personal gym-tracking web app for a small group of friends. Each athlete has a private space with their own exercise library, weekly plan, logged sets, gym attendance, body weight and InBody reports. It runs in the browser (React, Vite) on Firebase (Auth and Firestore) and is deployed on Vercel at no cost.

13	21	80	20	25
components (12 now, 1 planned)	use cases	functional requirements	non-functional requirements	UML diagrams

What the system does

- **Account:** sign up and sign in with email or Google; private data per athlete.
- **Exercise library:** 584 exercises in 17 muscle folders with images, steps, search, favorites, custom exercises and folders.
- **Weekly plan:** seven days (Saturday to Friday) with sets and reps targets; starter plans; shift, move, swap, copy and clear days with Undo.
- **Logging:** quick set logger with prefilled values, sets-done counter, PR announcement and rest timer.
- **Progress and attendance:** best weight, trend and PRs per exercise; gym days, weekly goal and week streak; Home dashboard.
- **Body:** weight goal with a barbell progress bar, weigh-ins, InBody reports with image and comparison, charts.

Architecture in one minute

The product is component-based, not monolithic. Components talk only through interfaces (a ball is a provided interface, a socket is a required one).

- **R1** One responsibility per component.
- **R2** One owner for every Firestore collection; others use the owner's interface.
- **R3** No circular dependencies (the data store gets the user id from the shell; analytics receives logs as arguments).
- **R4** Pages that combine components (Composition Views) contain no business rules.
- **R5** Only the data store and the sign-in code touch Firebase.

Documents in this package

- **Forge_Full_Documentation.pdf**: complete SRS, Component Specification, ownership note and all 25 UML figures.
- **Forge_SRS.docx** and **Forge_Component_Spec.docx**: editable Word versions.
- **docs/uml**: the diagram images and their sources.

The 13 components

ID	Component	Purpose	Provides	Requires
C1	**Identity & Profile**	Knows who the athlete is.	IAuth, IProfile	C2, Firebase Auth SDK
C2	**Data Store**	The only place that talks to Cloud Firestore.	IDataStore	Firestore SDK, Current uid
C3	**Exercise Library**	Everything about exercises as things: the 584 built-in exercises, how to find them, how they look, and the athlete's own exercise documents (favorites, custom exercises, folders)..	IExerciseCatalog, IExerciseSearch, Static data	C2, C10, Image CDN, YouTube
C4	**Workout Logging**	Records what the athlete lifted.	ILogger, ILogReader	C2, C3, C5, C10
C5	**Progress & Analytics**	Turns a list of logs into meaning: best weight, trend, PRs, sessions, and the small progress views.	IProgress	C10, C11
C6	**Weekly Plan**	The athlete's week: seven days (Saturday to Friday), each with a label, a rest flag, focus muscles and an ordered list of exercises with sets and reps targets, plus all the ways to reshape it safely..	IPlan	C2, C3, C5, C1, C11
C7	**Attendance & Goals**	Answers "did I show up?": which days count as gym days, the weekly goal and the week streak, and owns manual check-ins..	IAttendance	C2, C4, C6, C1, C10
C8	**Body Tracking**	Body weight over time and progress toward the weight goal..	IBodyTracker, IBodyReader	C2, C1, C10, C11
C9	**InBody Reports**	Body-composition scan reports: record, view, compare and chart them..	IInBody, IInBodyReader	C2, C10, C11
C10	**Units & Dates (Shared Kernel)**	Small, dependency-free rules every component shares: kilograms vs pounds, rounding, date strings and the Saturday-first week..	IUnits, IDates	None
C11	**UI Kit**	The design system as reusable parts.	UI components, Theme	Third-party libraries
C12	**App Shell & Composition Views**	Wires everything together: routing, navigation and the pages that combine several components.	Routes, Views	C1, C3, C4, C5, C6, C7, C8, C9, C11, C10
C13	**Crew & Leaderboard**	Opt-in sharing among friends with a fair ranking by percentage improvement.	ICrew	C5, C8, C1, C2

Who owns which data

Collection (users/{uid}/...)	Owner
(profile document)	**C1 Identity & Profile**
exercises, folders	**C3 Exercise Library**
logs	**C4 Workout Logging**
plan (and legacy schedule, read-only)	**C6 Weekly Plan**
checkins	**C7 Attendance & Goals**
weights	**C8 Body Tracking**
inbody	**C9 InBody Reports**

Requirements at a glance

80 functional requirements (60 must-have) and 20 non-functional requirements. Every requirement links to a use case and a component.

ID	Component	Reqs	Must	Range
C1	Identity & Profile	7	7	FR-ID-01 to FR-ID-07
C2	Data Store	4	3	FR-DS-01 to FR-DS-04
C3	Exercise Library	12	7	FR-LIB-01 to FR-LIB-12
C4	Workout Logging	9	6	FR-LOG-01 to FR-LOG-09
C5	Progress & Analytics	6	3	FR-PRG-01 to FR-PRG-06
C6	Weekly Plan	13	10	FR-PLN-01 to FR-PLN-13
C7	Attendance & Goals	6	6	FR-ATT-01 to FR-ATT-06
C8	Body Tracking	4	4	FR-BODY-01 to FR-BODY-04
C9	InBody Reports	5	2	FR-INB-01 to FR-INB-05
C10	Units & Dates (Shared Kernel)	3	3	FR-UNT-01 to FR-UNT-03
C11	UI Kit	6	4	FR-UI-01 to FR-UI-04
C12	App Shell & Composition Views	5	5	FR-APP-01 to FR-APP-05

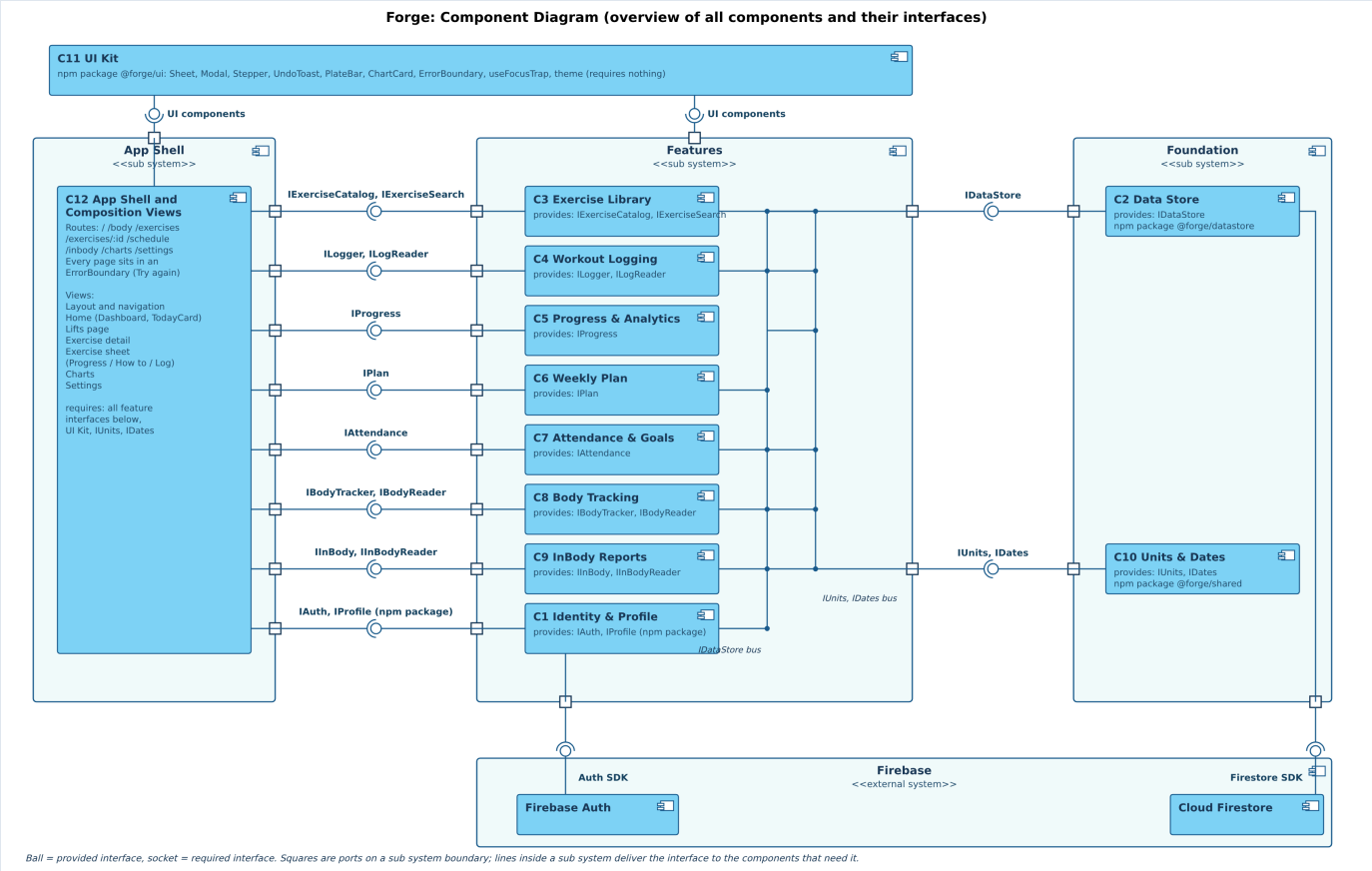
Quality targets (examples)

- **Performance:** The first screen after sign-in shall be interactive within 3 seconds on a mid-range phone over a 4G connection; the exercise library (about 600 KB) shall load on demand in a separate bundle.
- **Security & privacy:** Each user shall be able to read and write only the documents under their own users/{uid} path, enforced by Firestore security rules.
- **Usability & accessibility:** The interface shall be mobile-first; every tap target shall be at least 44 x 44 px.
- **Reliability & data integrity:** No plan operation shall delete logs, exercises or the legacy schedule.
- **Maintainability:** The code shall be organised as the components in Section 3 (Identity, Data Store, Units & Dates and UI Kit as npm workspace packages) with one owner per Firestore collection, packages used only through their entry point, and no circular dependencies.
- **Portability & compatibility:** The app shall run in current versions of Chrome, Safari, Firefox and Edge on phones and desktops.
- **Cost & deployment:** The system shall run within the free tiers of Firebase (Spark plan), Vercel and the image CDN; no paid service and no Firebase Storage shall be required.

Key business rules

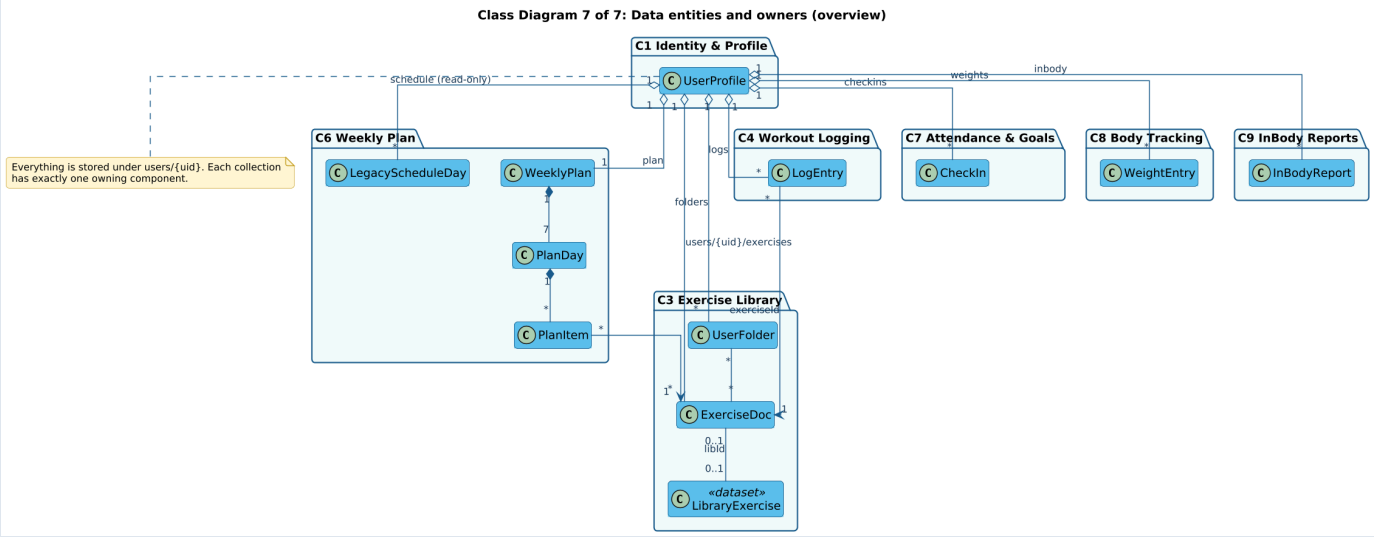
- All weights are stored in kg; dates are YYYY-MM-DD; weeks start on Saturday.
- Progress % = (start - current) / (start - target), clamped to 0-100, for loss or gain.
- A PR is a set heavier than every earlier set of that exercise.
- A gym day is a logged set or a manual check-in; weekly goal = non-rest plan days with exercises (default 3 without a plan).
- Deleting a folder or a plan day never deletes exercises or logs; plan changes are undoable.

Component diagram



Sub systems, ports and provided/required interfaces of all components.

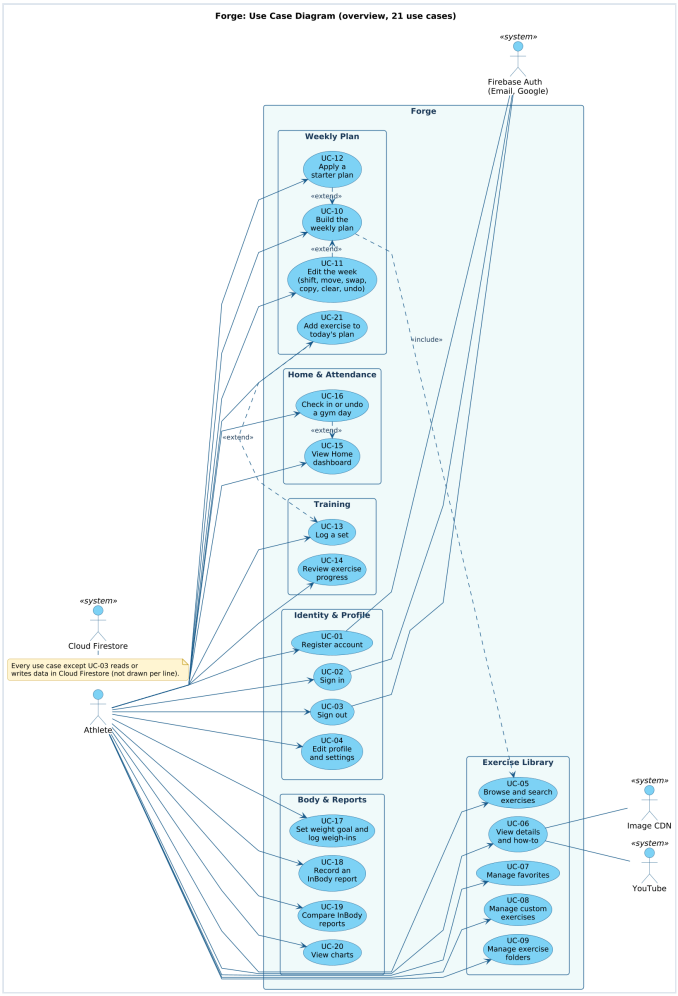
Data entities and owners



Every collection belongs to exactly one component.

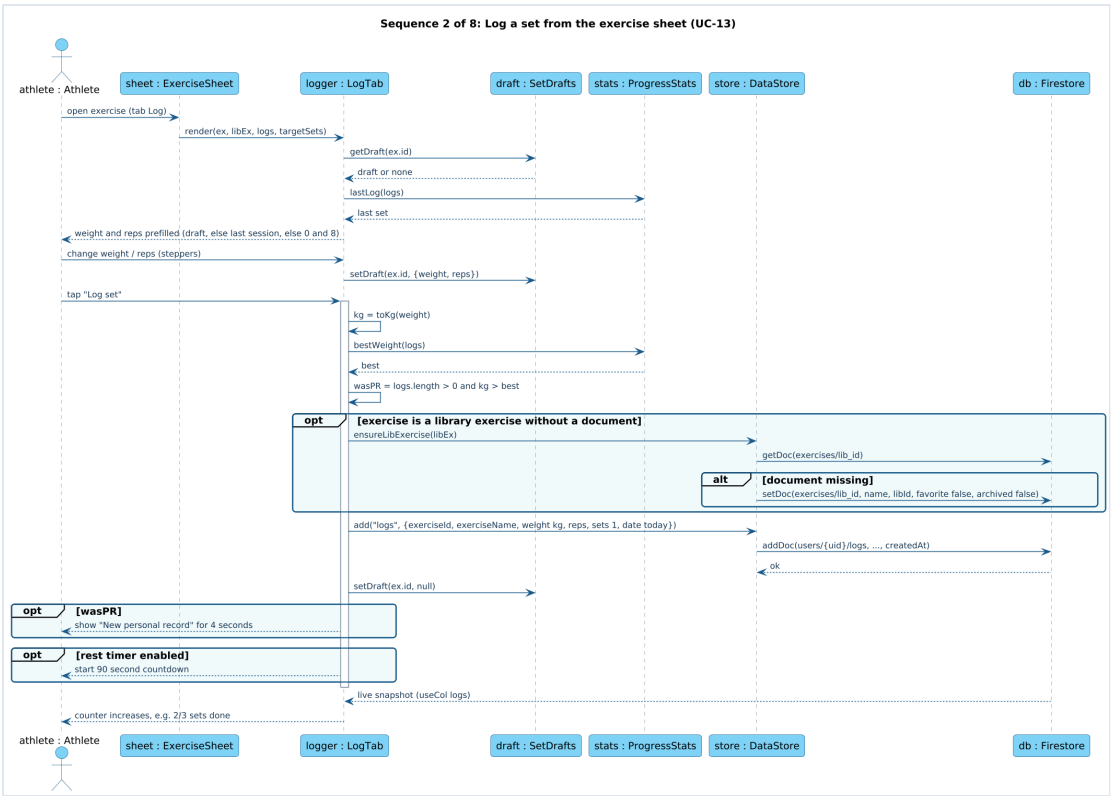
Use cases

ID	Use case
UC-01	Register account
UC-02	Sign in (email or Google)
UC-03	Sign out
UC-04	Edit profile and settings
UC-05	Browse and search exercises
UC-06	View exercise details and how-to
UC-07	Manage favorites
UC-08	Manage custom exercises
UC-09	Manage exercise folders
UC-10	Build the weekly plan
UC-11	Edit the week (shift, move, swap, copy, clear, undo)
UC-12	Apply a starter plan
UC-13	Log a set
UC-14	Review exercise progress
UC-15	View Home dashboard
UC-16	Check in or undo a gym day
UC-17	Set weight goal and log weigh-ins
UC-18	Record an InBody report
UC-19	Compare InBody reports
UC-20	View charts
UC-21	Add an exercise to today's plan

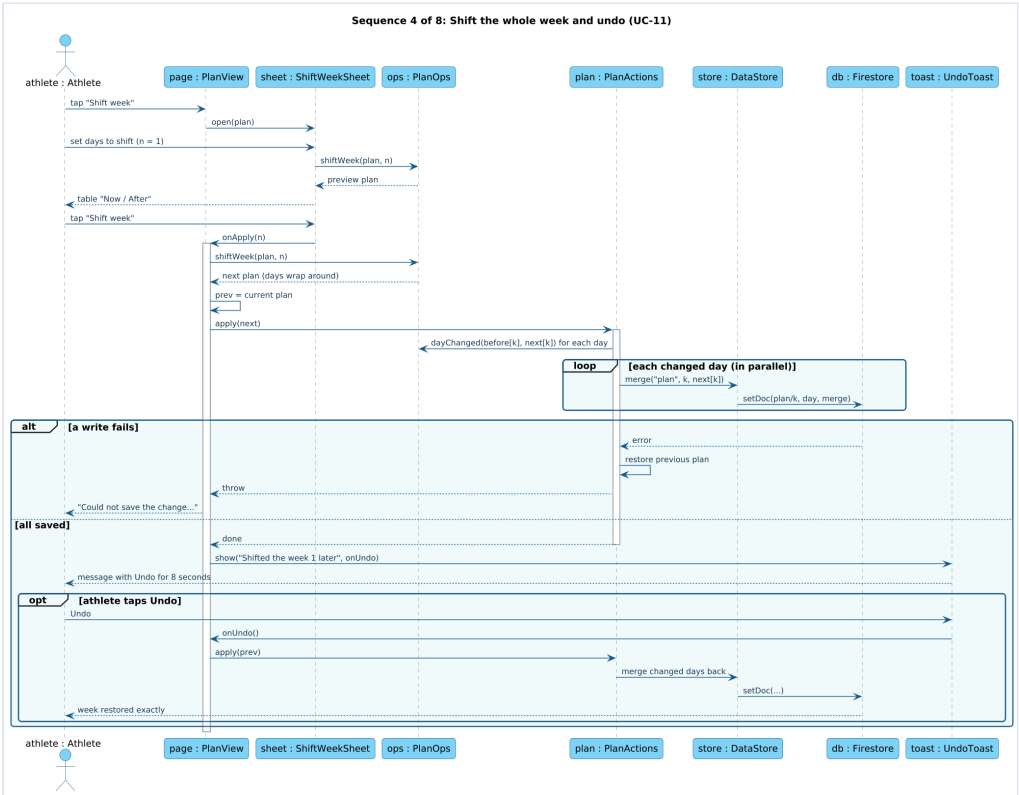


Use case diagram (overview)

Key flows



Log a set (UC-13): prefill, PR check, lazy creation of the exercise document, live counter.



Shift the whole week with Undo (UC-11): writes only the changed days.

Status and next steps

Design and code now match. The code was refactored into the component folders described in the documents. The running app has the same features as before. Notes:

- AppContext.jsx and utils.js are gone. Four components (identity, datastore, shared, ui) are npm workspace packages in packages/ and can be reused in other projects; the rest are folders of the app.
- Cross-component reads go through owner hooks (useLogs, useExerciseDocs, useWeights, useReports); logs are passed into the library and plan components.
- An automated architecture test checks the allowed dependencies, the Firebase boundary and the absence of cycles. Every page sits in an error boundary, so a crash in one page does not blank the app.
- Small leftovers: chart series are built inside the Charts view, and undo orchestration is inside the Plan view.

Recommended next steps

- Test the preview deployment, then merge the refactor.
- Add the crew leaderboard (C13) as the first new component, using the same interfaces.
- Optional clean-ups: move chart series into progress/ and undo orchestration into plan/.